RBB3023
Embedded Systems

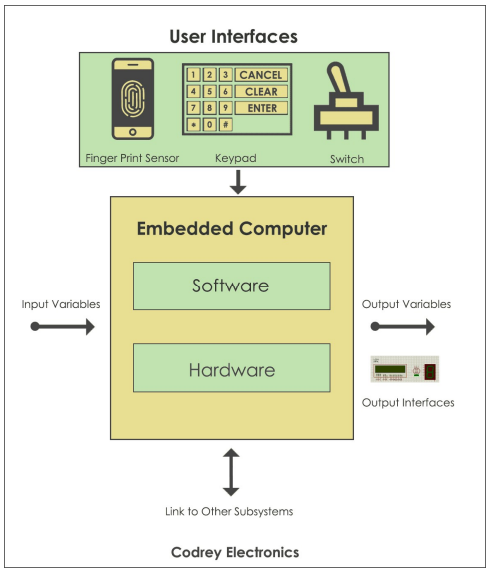
Semester May 2026

Patrick Sebastian

1

Various definitions of "embedded system"

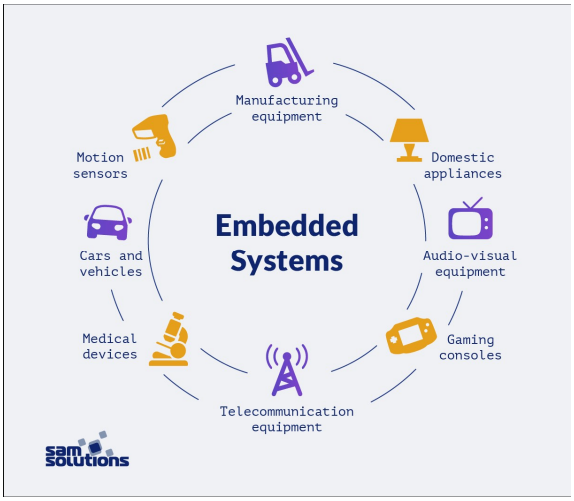
- **An embedded system is a computer system—a combination of a computer processor, computer memory, and input/output peripheral devices—that has a dedicated function within a larger mechanical or electrical system.**
  - (Wikipedia)



2

Various definitions of "embedded system"

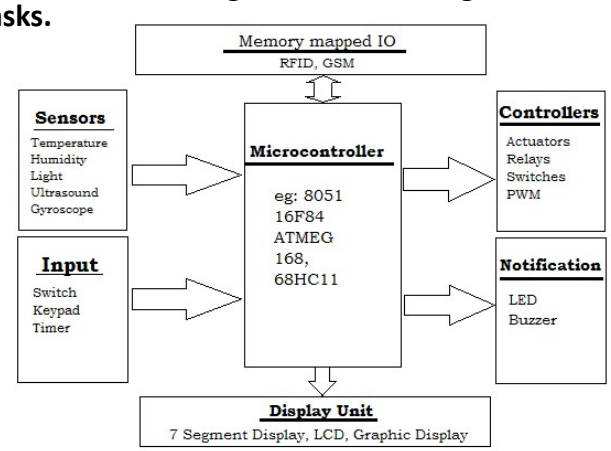
- **An embedded system is a combination of computer hardware and software designed for a specific function or functions within a larger system. The systems can be programmable or with fixed functionality.**
  - Industrial machines, consumer electronics, agricultural and process industry devices, automobiles, medical equipment, cameras, household appliances, airplanes, vending machines and toys, as well as mobile devices, are possible locations for an embedded system.
  - (IoT Agenda)



3

Various definitions of "embedded system"

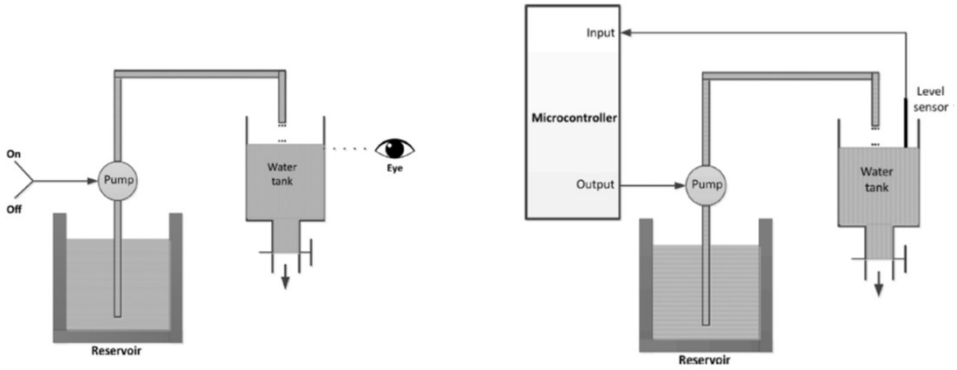
- **An embedded system is a dedicated computer system designed for one or two specific functions. This system is embedded as a part of a complete device system that includes hardware, such as electrical and mechanical components. The embedded system is unlike the general-purpose computer, which is engineered to manage a wide range of processing tasks.**
  - (Techopedia)



4

Microcontroller-based control system

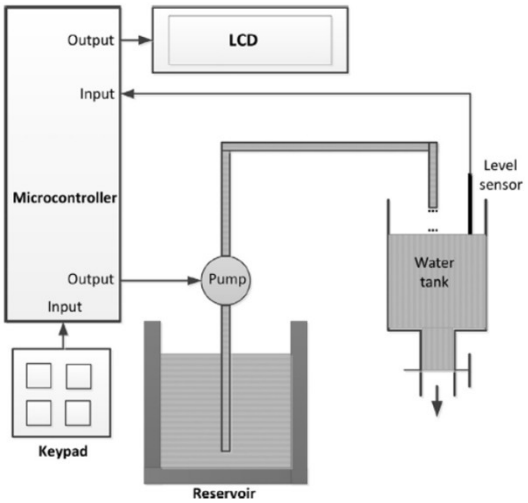
- **Manual vs microcontroller-based liquid level control systems**



5

Microcontroller-based control system

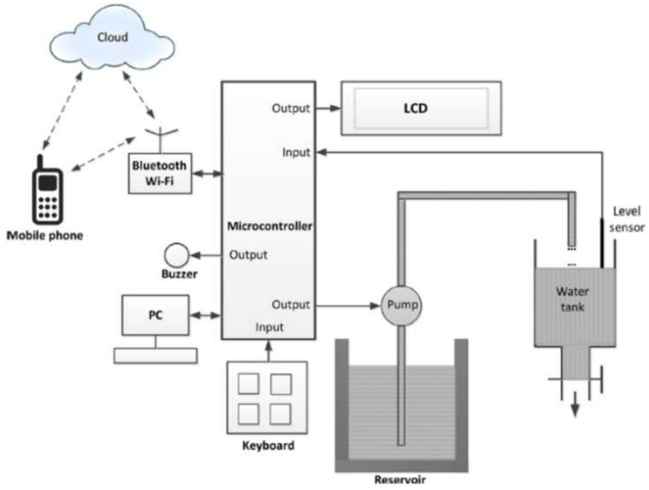
- **Microcontroller-based liquid level control system with input/output devices for the user interaction**



6

Microcontroller-based control system

- **Microcontroller-based liquid level controller system as part of internet of things (IoT)**



7

Difference between firmware, embedded system and IOT

Firmware	Embedded Systems	IOT
Firmware was software stored in special memory such as EEPROM	Embedded software is software designed for particular device. Normally runs on a microcontroller	An extension of the embedded system that has a connection to the Internet.
The software stored in these devices are permanent and immoveable : “firmware”	Program resides in file system.	Has networking ability
Typically used as low level software that operates a specific single purpose device. A lot of systems use a lot of firmware to control different devices.	Implements higher level features and functions.	
Language typically used are assembly and C	Languages are highly sophisticated such as C++, C, etc etc. Can use an Operating System (RTOS)	

8

Overview of ARM Processors

Learning outcome:

CLO1

Develop programs in Assembly and C to perform specific tasks (PO2)

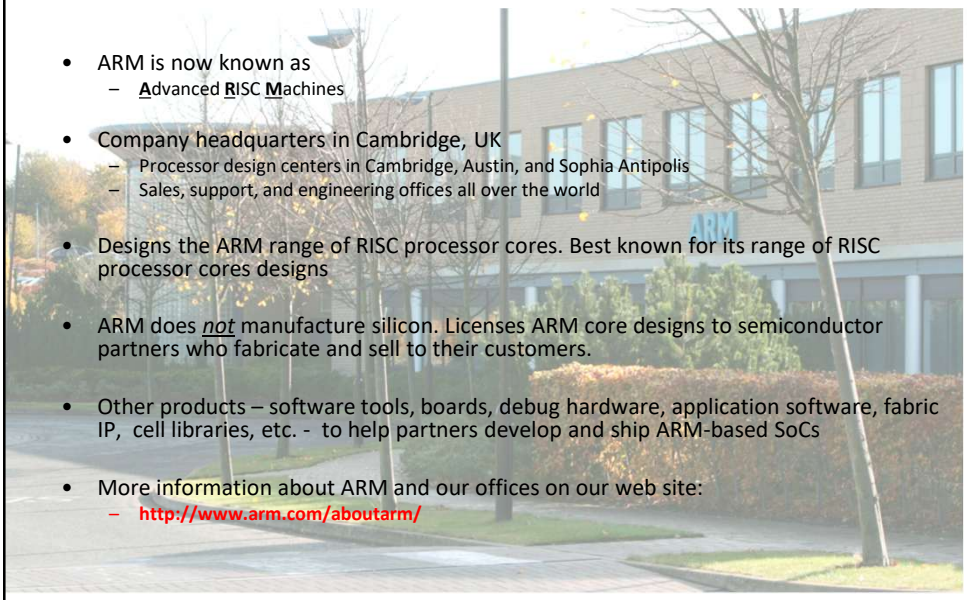
9

The ARM Processor



10

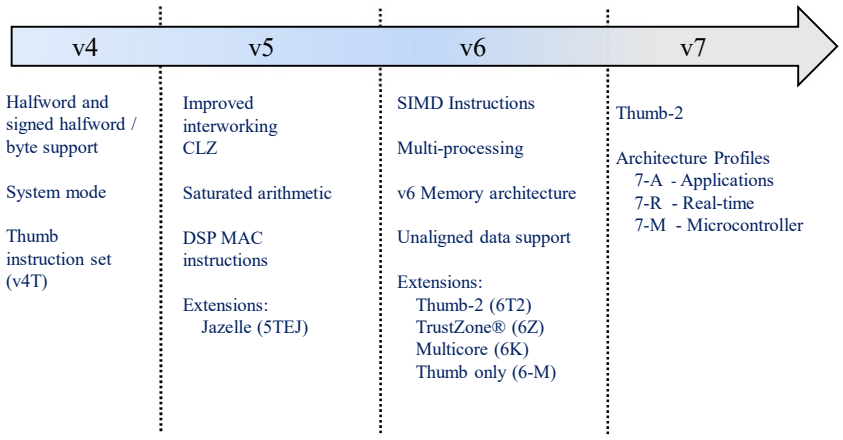
ARM Ltd



- ARM is now known as
  - **A**dvanced **R**ISC **M**achines
- Company headquarters in Cambridge, UK
  - Processor design centers in Cambridge, Austin, and Sophia Antipolis
  - Sales, support, and engineering offices all over the world
- Designs the ARM range of RISC processor cores. Best known for its range of RISC processor cores designs
- ARM does *not* manufacture silicon. Licenses ARM core designs to semiconductor partners who fabricate and sell to their customers.
- Other products – software tools, boards, debug hardware, application software, fabric IP, cell libraries, etc. - to help partners develop and ship ARM-based SoCs
- More information about ARM and our offices on our web site:
  - **http://www.arm.com/aboutarm/**

11

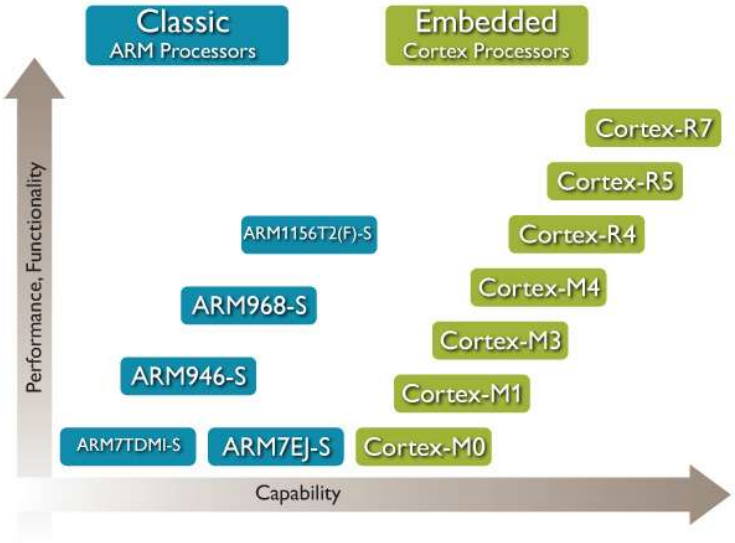
ARM Architecture Development



- **Note that implementations of the same architecture can be different**
  - Cortex-A8 - architecture v7-A, with a 13-stage pipeline
  - Cortex-A9 - architecture v7-A, with an 8-stage pipeline

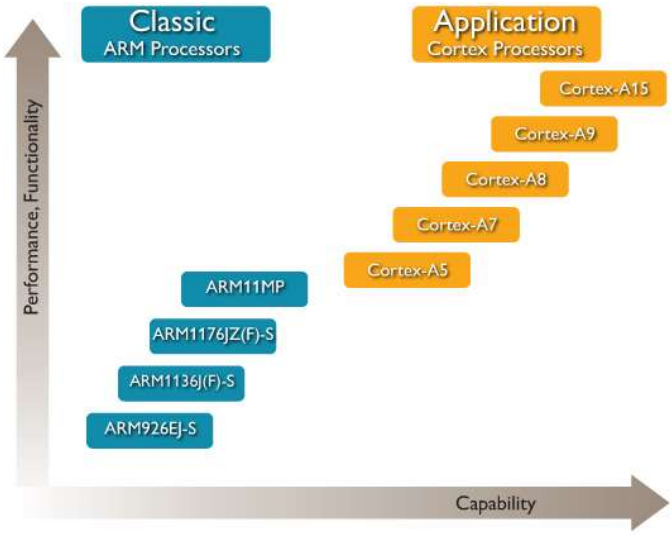
12

Embedded Processors



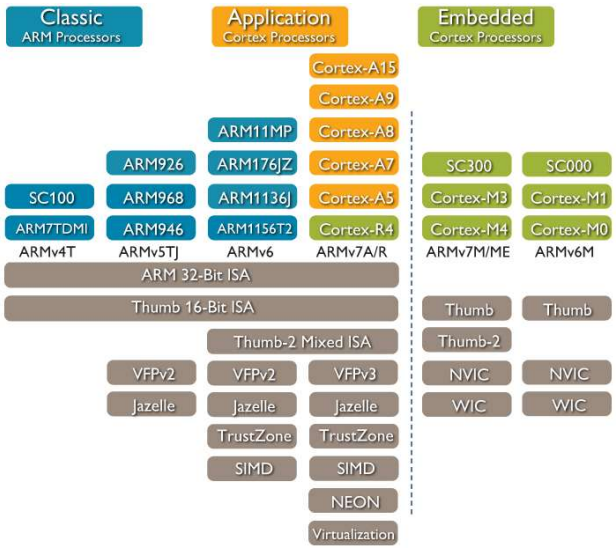
13

Application Processors



14

Which Architecture is My Processor? [2]



15

Which Architecture is My Processor? [2]

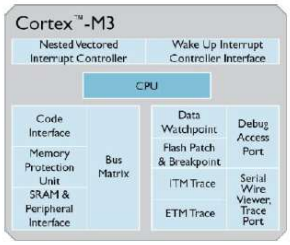
	Application processors	Real-Time processors	Microcontroller processors
Design	High clock frequency, Long pipeline, High performance, Multimedia support (NEON instruction set extension)	High clock frequency, Long to medium pipeline length, Deterministic (low interrupt latency)	Short pipeline, ultra low power, Deterministic (low interrupt latency)
System features	Memory Management Unit (MMU), cache memory, TrustZone® security extension	Memory Protection Unit (MPU), cache memory, Tightly Coupled Memory (TCM)	Memory Protection Unit (MPU), Nested Vectored Interrupt Controller (NVIC), Wakeup Interrupt Controller (WIC)
Targeted markets	Mobile computing, smart phones, energy efficient servers, high end microprocessors	Industrial microcontrollers, automotives, Hard disk controllers, Baseband modem.	Microcontrollers, Deeply embedded systems (e.g. sensors, MEMS, mixed signal IC), Internet of Things (IoT)

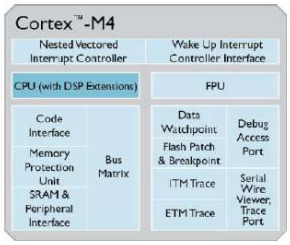
Table 1: Summary of processor characteristics

16

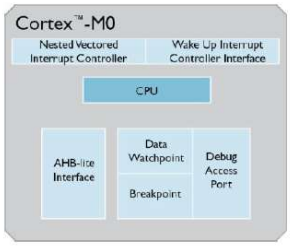
Cortex-M processors

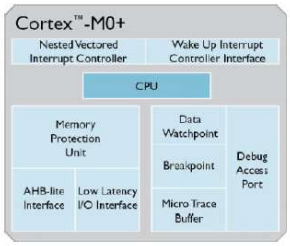
ARMv7-M Processors





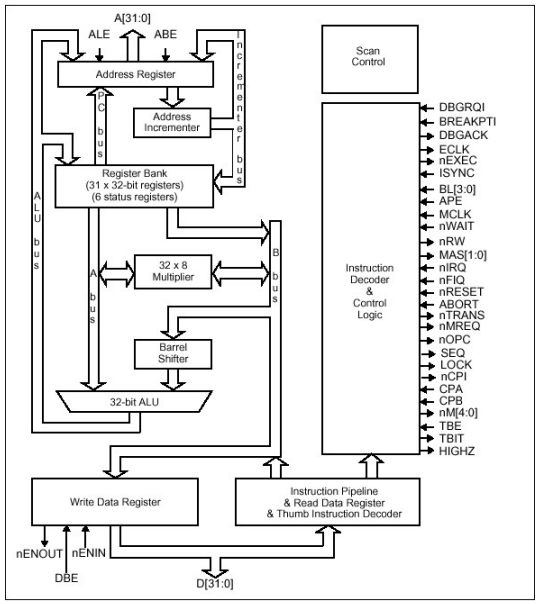
ARMv6-M Processors





17

ARM Core Diagram - ARM7TDMI core

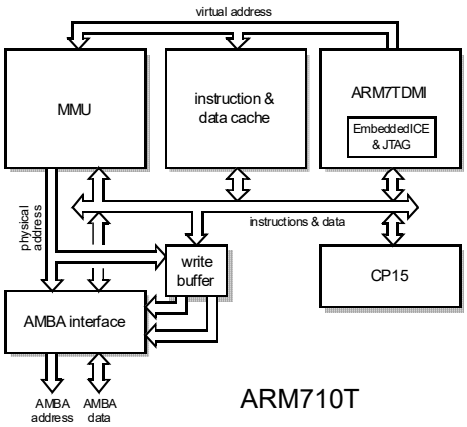


Internal

18

Processor Core vs MPU/MCU Core

- **Processor Core**
  - The engine that fetches instructions and execute them
  - E.g.: ARM7TDMI, ARM9TDMI, ARM9E-S
- **MPU/MCU Core**
  - Consists of the ARM processor core and some tightly coupled function blocks
  - Cache and memory management blocks
  - E.g.: ARM710T, ARM720T, ARM74T, ARM920T, ARM922T, ARM940T, ARM946E-S, and ARM966E-S



ARM710T

Internal

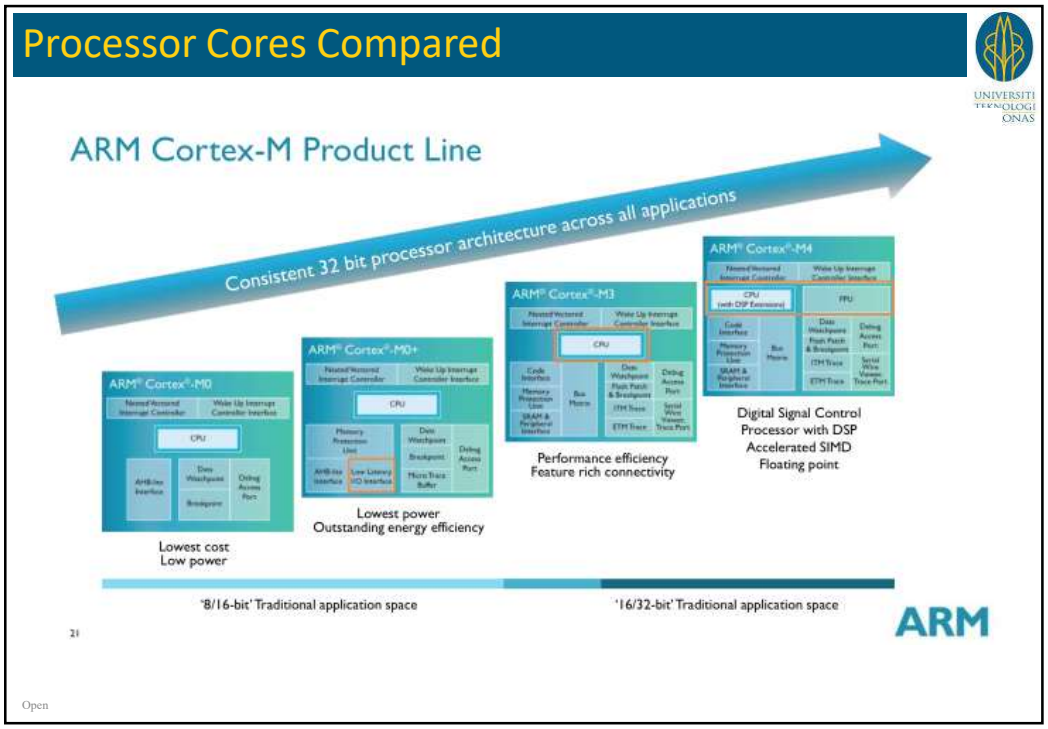
19

Processor Cores Compared

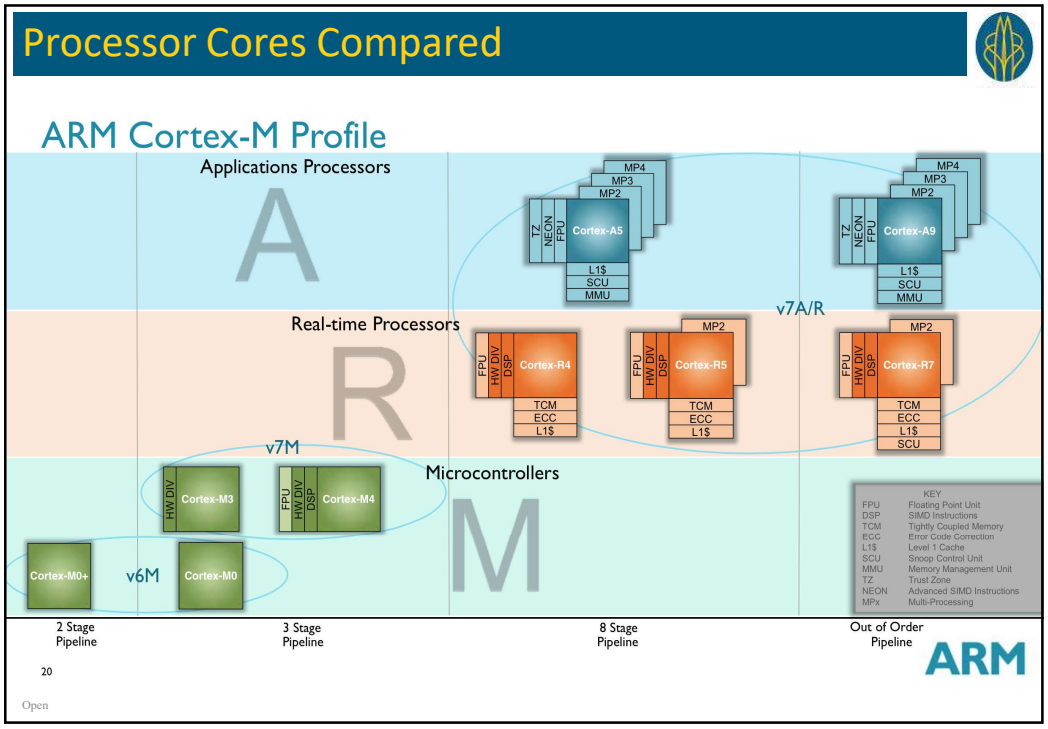
	ARM7	ARM9	ARM10	ARM11
Pipeline depth (stages)	3	5	6	8
Typical speed (MHz)	80	150	260	335
mW/Mhz	0.06	0.19	0.50	0.40
Cache		+cache	+cache	+cache
MIPS/Mhz	0.97	1.10	1.30	1.20
Architecture	Von Neumann	Harvard	Harvard	Harvard
Multiplier	8 x 32	8 x 32	16 x 32	16 x 32

Open

20

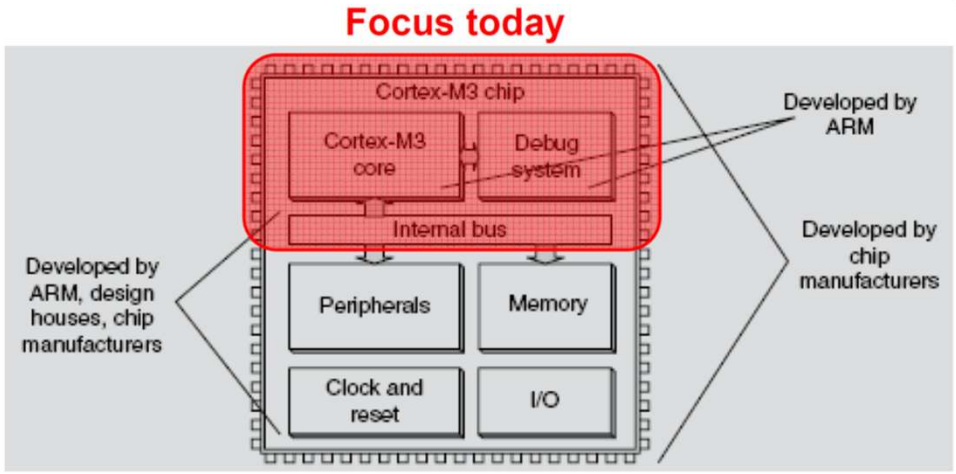

Processor Cores Compared
ARM Cortex-M Product Line
Consistent 32 bit processor architecture across all applications
ARM® Cortex®-M0
ARM® Cortex®-M0+
ARM® Cortex®-M3
ARM® Cortex®-M4
Lowest cost
Low power
Lowest power
Outstanding energy efficiency
Performance efficiency
Feature rich connectivity
Digital Signal Control
Processor with DSP
Accelerated SIMD
Floating point
'8/16-bit' Traditional application space
'16/32-bit' Traditional application space
ARM
21
Open

21


Processor Cores Compared
ARM Cortex-M Profile
Applications Processors
A
Real-time Processors
R
Microcontrollers
M
v7A/R
v7M
v6M
MP4
MP3
MP2
TZ
NEON
FPU
Cortex-A5
Cortex-A9
L1$
SCU
MMU
FPU
HW DIV
DSP
Cortex-R4
Cortex-R5
Cortex-R7
TCM
ECC
L1$
Cortex-M3
Cortex-M4
Cortex-M0+
Cortex-M0
KEY
FPU Floating Point Unit
DSP SIMD Instructions
TCM Tightly Coupled Memory
ECC Error Code Correction
L1$ Level 1 Cache
SCU Snoop Control Unit
MMU Memory Management Unit
TZ Trust Zone
NEON Advanced SIMD Instructions
MPx Multi-Processing
2 Stage Pipeline
3 Stage Pipeline
8 Stage Pipeline
Out of Order Pipeline
ARM
20
Open

22

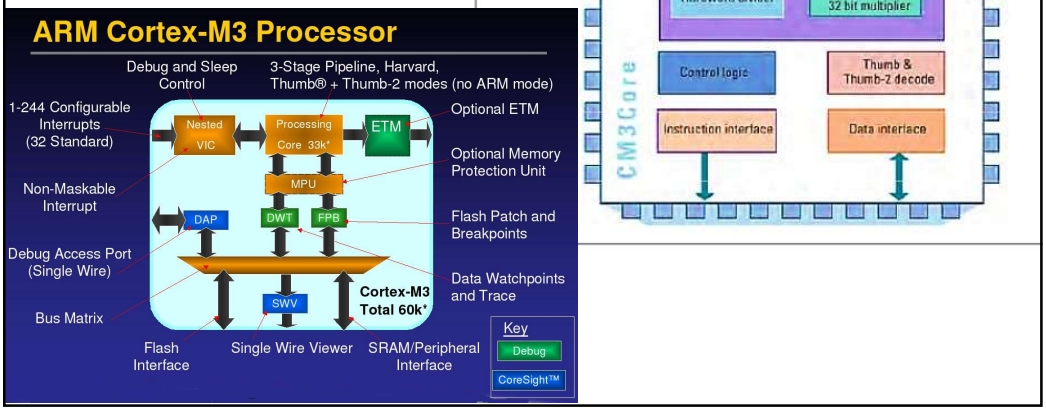
Functional Units of ARM Processors



Processor vs. MPU (Microprocessor Unit)/ MCU (Microcontroller Unit)

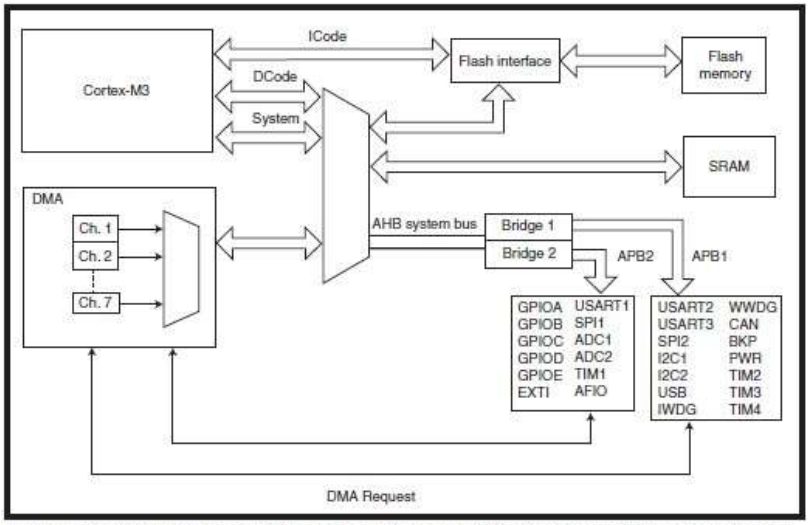
23

Cortex-M3 Processor Core



24

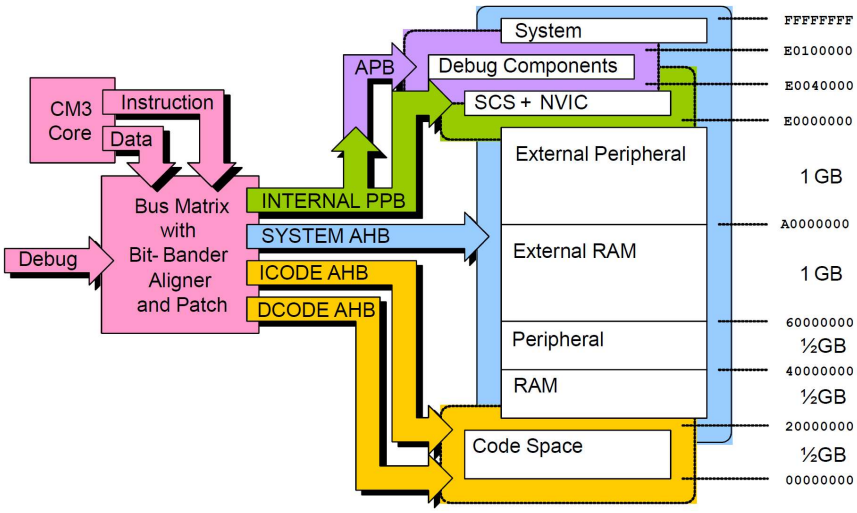
Interconnections between Cortex-M3 Core and Memory, I/O Peripherals



25

Memory Map (Cortex-M3/M4)

- Very simple linear 4GB memory map
- The Bus Matrix partitions memory access via the AHB and PPB buses

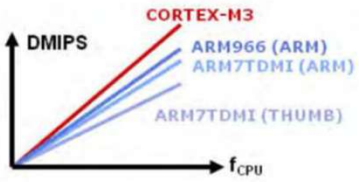


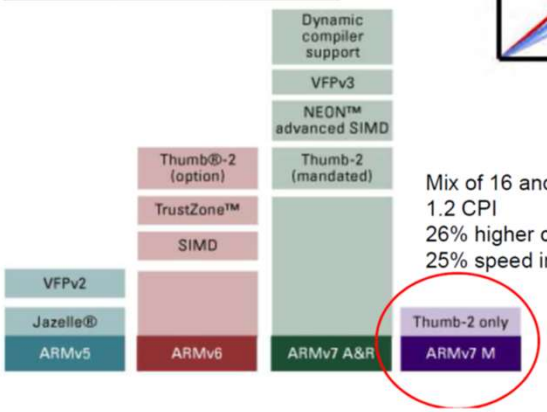
26

Which Instruction(s) are used by My Processor?

ARMv7 M (Thumb-2) features

Source	Destination	Cycles
16b x 16b	32b	1
32b x 16b	32b	1
32b x 32b	32b	1
32b x 32b	64b	3-7*



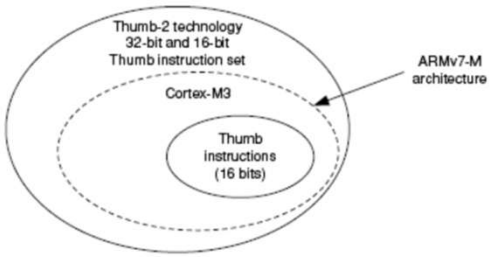


Mix of 16 and 32b instructions
1.2 CPI
26% higher code density ARM32
25% speed improvement over Thumb16

27

Thumb-2

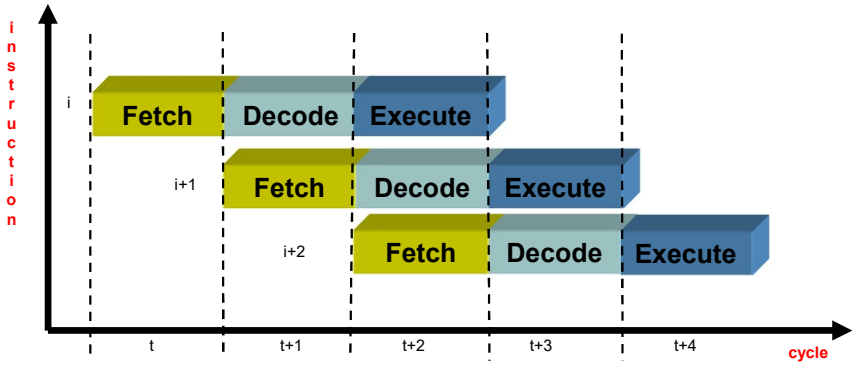
- Mixes 16 and 32 bits instructions
  - Enhancements: eg. UDIV, SDIF division, bit-field operators UFBX, BFC, BFE, wrt traditional ARMv4T
  - No need to mode switch, can be mixed freely
- **Not** backwards binary compatible
  - But porting is «easy»



28

Pipeline Organization for Cortex-M3 [1]

- 3-stage pipeline: Fetch – Decode - Execute
- Three-cycle latency → one instruction per cycle throughput



29

Pipeline Organization for Cortex-M3 [2]

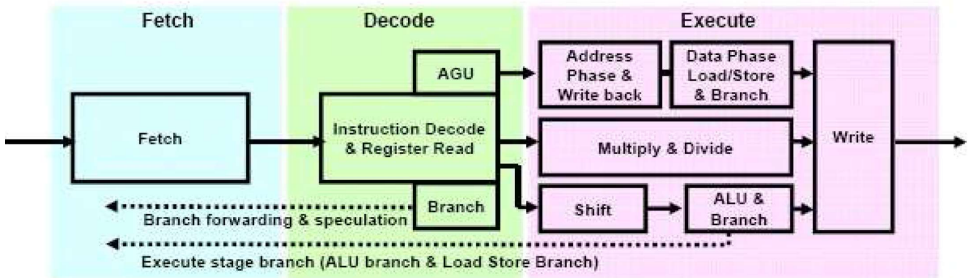
Harvard Architecture

- Separate instruction (code) and data buses
- Enables parallel fetch and store

Advanced 3-Stage Pipeline

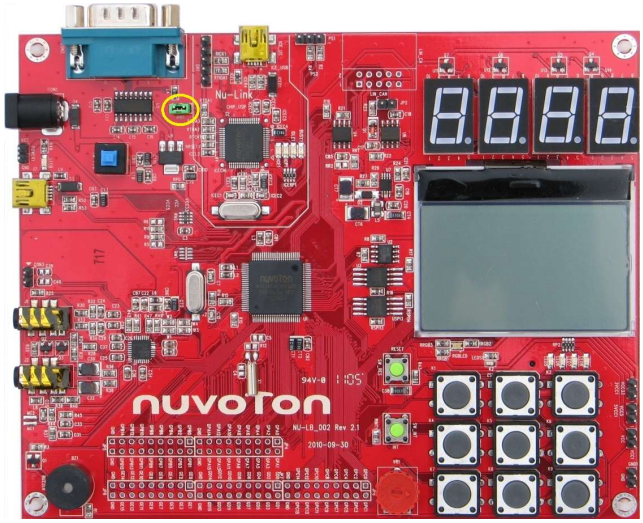
- Includes Branch Forwarding and Speculation

Additional Write-Back via Bus Matrix



30

ARM Start-Up Kit [1]: NUVOTON LEARNING BOARD (Cortex M0+)



31

NuMicro™ A Family of McU Powered by ARM Cortex-M

Future products with higher density and more functionalities

Up to512KBFlash ROM	Up to64KBSRAM	1x12-bit ADC	Ethernet MAC	Motor PWM

NUC140 Connectivity Line				
Up to 128KB Flash ROM	Up to16KB SRAM	CAN	LIN	USB
NUC130 Automotive Line				
Up to 128KB Flash ROM	Up to16KB SRAM	CAN	LIN	
NUC120 USB Line				
Up to 128KB Flash ROM	Up to16KB SRAM	USB		
NUC100 Advance Line				
Up to 128KB Flash ROM	Up to16KB SRAM			
M051 Base Line				
Up to 64KB Flash ROM	Up to4KB SRAM			

∗ : Denoted not supported in M051 series

Cortex-M0	
UART	
SPI	
I²C	
I²S	∗
8-channel PWM	
12-bit ADC	
Comparators	∗
32-bit Timers	
Watchdog	
Internal 22MHz RC	
Real Time Clock	∗
PDMA	∗
GPIO	

32

NUC130/140 Key Features

- ARM® Cortex™-M0 core
  - Up to 50MHz
  - Single-cycle 32-bit Multiplier
  - NVIC suppor 32 interrupts with 4 level priorities
- Flash EPROM Memory
  - 128K bytes Flash EPROM.
  - 16K bytes SRAM
  - 4kB ISP loader flash
- Communication Interfaces
  - UART
  - SPI
  - I2C (up to 1MHz)
- Analog Peripheral
  - 8 channels of 12-bit ADC800Ksps
- Connectivity Peripherals
  - USB FS 2.0
  - LIN Bus
  - CAN 2.0B --
- Interfaces
  - IrDA
  - PWM
  - RTC
  - Comparators
  - Brown-out Detect
  - GPIO
  - DMA
  - 4 x 32-bit Timer
- Power Management
  - Idle Mode
  - Power down Mode

33

33

NUC130/140 Key Features (cont.)

- Operating Voltage : 2.5 ~ 5.5V
  - Ideal for Automotive and Industrial control
- Temperature : -40℃ ~ 85℃
- AEC-Q100 (Grade 3)
- High ESD & EFT,
- Internal RC oscillator with 1% accuracy @room temp
- On-chip lock bit with security capability
- Embedded data-flash
- Brown-out Detection
- High performance interface

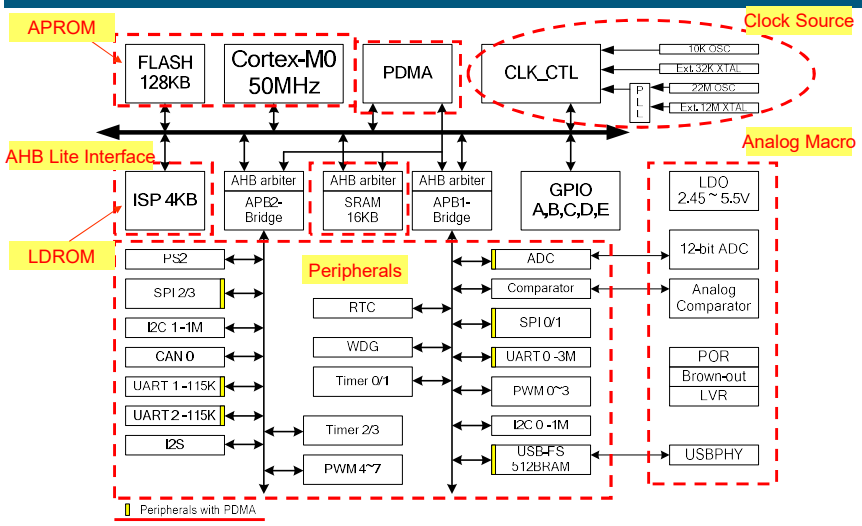
UART	3Mbps
IrDA	115200bps
SPI	32MHz
I2C	1MHz
USB FS 2.0	12Mbit/s
CAN 2.0B	1M bit/s
LIN	3M bps



34

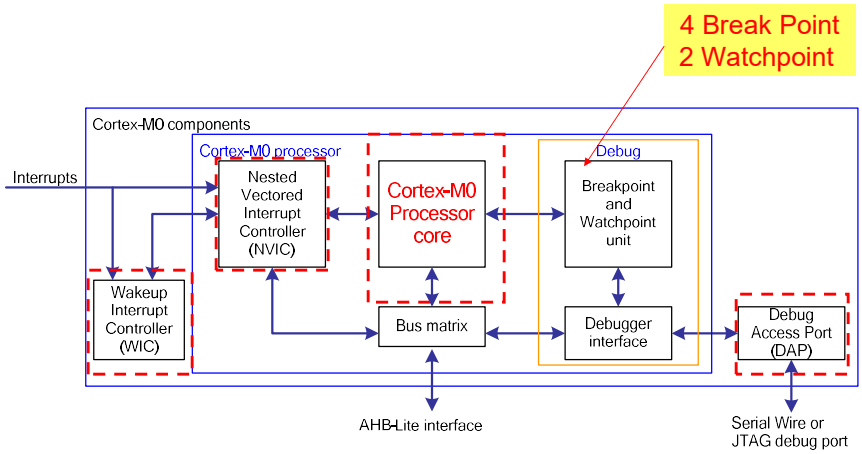
34

NUC100 Series MCU Block Diagram



35

ARM Cortex-M0 Functional Block Diagram



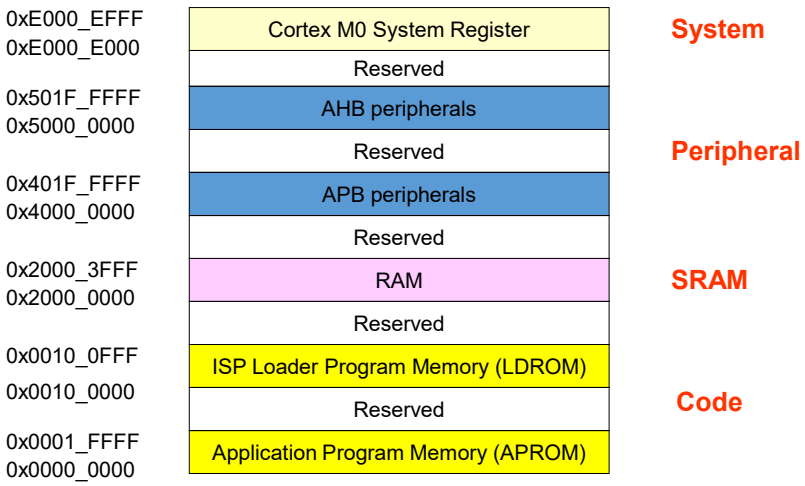
36

NuMicro Family System Introduction

- **The NUC 100 series IC embed Cortex –M0 core**
  - running up to 50 MHz
  - 32K/64K/128K bytes embedded flash by part No
    - **Configurable data flash address and size for 128kB system, fixed 4kB data flash for the 32kB and 64kB system**
  - 4kB flash for ISP loader
  - 4K/8K/16K embedded SARM
- **The M051 series IC embed Cortex-M0 core**
  - 8KB/16KB/32KB/64KB Flash memory for program memory (APROM)
  - 4KB Flash memory for data memory (DataFlash)
  - 4KB Flash memory for loader (LDROM)
  - 4KB SRAM for internal scratch-pad RAM (SRAM)
- **Support ISP (In System Programming) by USB or UART**
- **Support 2 wire ICP (In Circuit Programming) update from ICE interface**
- **Support fast parallel programming mode by external writer**
- **2 wire SWD ICE interface**
- **Wide operating voltage ranges from 2.5V to 5.5V**

37

System Memory Map for Nuvoton



38

Nucleo-F411RE development board

- **STM32 Nucleo-64 development board**
  - with STM32F411RE MCU, supports Arduino and ST morpho connectivity
- **Features**
  - STM32 microcontroller in LQFP64 package
  - 1 user LED, 1 user and 1 reset push-buttons
  - 32.768 kHz crystal oscillator
  - Arduino™ Uno V3 expansion connector
  - ST morpho extension pin headers full access to all I/Os
  - On-board ST-LINK debugger/programmer
  - Arm® Mbed Enabled™ compliant



https://www.st.com/en/evaluation-tools/nucleo-f411re.html

Internal 39

39

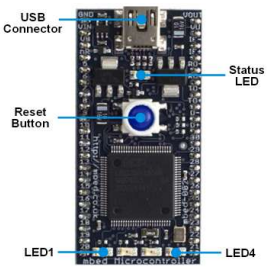
NXP LPC1768 development board

Serial Peripherals

- 10/100 Ethernet MAC
- USB 2.0 full-speed device/Host/ OTG controller with on-chip PHY
- Four UARTs with fractional baud rate generation, RS-485. modem control I/O, IrDA
- Two CAN 2.0B controllers
- Three SSP/SPI controllers
- Three I2C-bus interfaces with one supporting fast mode plus (1 Mbit/s data rates)
- I2S interface for digital audio

Other Peripherals

- Real-Time Clock operating at < 1 uA
- Eight channel General Purpose DMA controller
- Up to 70 General Purpose I/O
- Motor control PWM and Quadrature Encoder Interface to support three-phase motors
- Four 32-bit general purpose timers/counters
- 4 MHz internal RC oscillator trimmed to 1 % accuracy



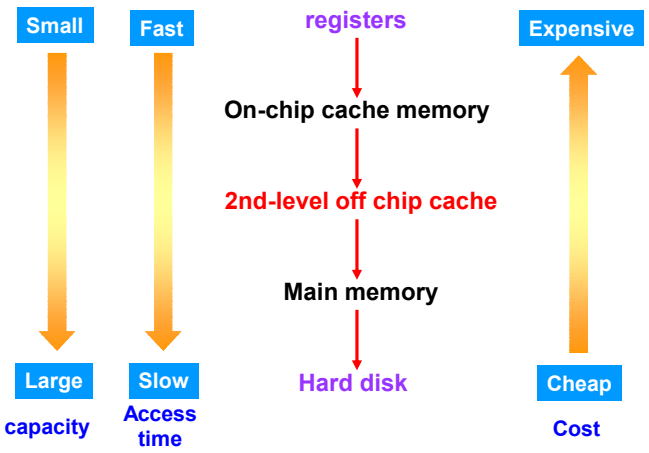
Internal 40

40

Memory organization

41

Memory Hierarchy, Size and Speed [1]



42

Memory Hierarchy, Size and Speed [2]

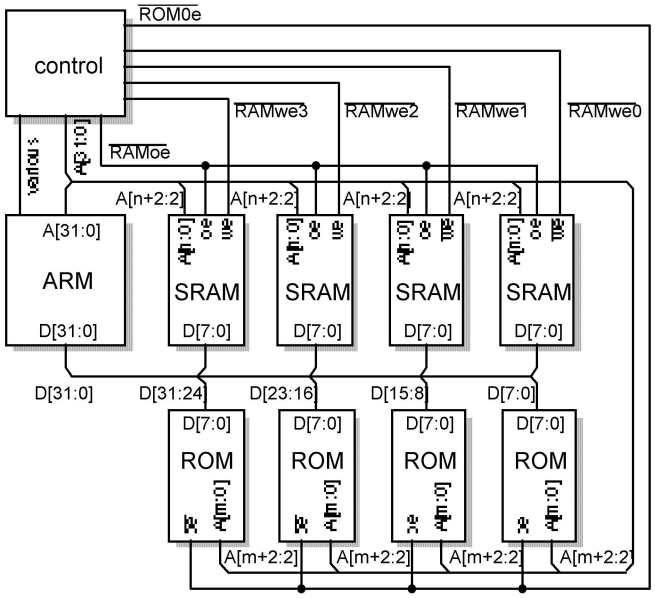
Larger size ⟺ Lower speed

Memory type	Size	Speed
Registers	32 bits	A few nsec
On – chip cache	8 – 32 kbytes	10 nsec
Off – chip cache	100 – 200 kbytes	10 – 30 nsec
RAM	Mbytes	100 nsec

43

The ARM Memory Interface

A basic ARM memory system



44

Memory Concept [1]

There are various ways of depicting memory; a common one is a 'hex dump':

Address	Values as 8-bit numbers	Characters
00000000	01 99 34 26 45 AE 63 3F	..4&E.c?
00000008	48 65 6C 6C 6F 00 00 00	Hello...
…….. .. .. ……..		

You will see things like this in the labs. Each item has a unique address – not all are shown here.

The values are open to interpretation. You do not need to worry about any other details for now.

45

Memory Concept [2]

- Program code and data are saved in memory
- Program code and data occupy different memory areas

Four 8-bit data form a 32-bit word

Labels	32-bit Address (HEX)	8-bit contents (Program/Data)	
Org	0000 0000	03	
	0000 0001	24	Program
	:		
TABLE1	0001 0000	12	
	0001 0001	3B	
	0001 0002	A4	
	0001 0003	34	
	0001 0004	B2	Data
	0001 0005	D2	
:			big/little endian formats
TABLE2	0002 0000	24	
	0002 0001	6C	
:			

46

Sample of Contents inside Memory



- Memory is a linear array of bytes from 0 (0x0000 0000) to 2^{32} – 1 (0xFFFF FFFF)
- Each byte has an address
- Word (32 bits); half-word (16 bits), byte (8 bits)

0xFFFFFFFF	00
0xFFFFFFFE	00
0xFFFFFFFD	00
0x00000006	FF
0x00000005	FF
0x00000004	FF
0x00000003	30
0x00000002	20
0x00000001	10
0x00000000	00

47

Sample of Contents inside Registers and Memory



r0	01111110100000000000000000000000
r1	10000000000000011100101010010010
r2	11000000000000011100101010010010
.....	
r13	11111000000000011100101010010010
r14	00000011110000011100101010010010

cpsr	NZCV	IFT43210

0xFFFFFFFF	00
0xFFFFFFFE	00
0xFFFFFFFD	00
0x00000006	FF
0x00000005	FF
0x00000004	FF
0x00000003	30
0x00000002	20
0x00000001	10
0x00000000	00

48

Register and Memory Access [1]



- In the ARM architecture, memory is byte addressable
  → 32-bit addresses (Memory is addressed as a 32 bit address space)
  → 32-bit processor registers

- ARM processor uses load/store architecture, i.e.,
  → Only 32 bit data bus for both instruction and data
  → Only the load/store instruction access memory

- When a byte is loaded from memory into a processor register or stored
  from a register into the memory, it is always located in the low-order byte position of the register

49

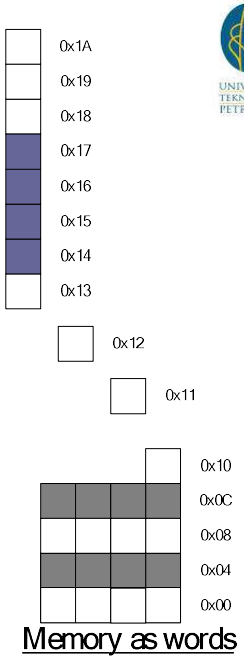
Register and Memory Access [2]



- Data type can be 8 bit bytes, 16 bit half-words or 32 bit words, and may be seen as a byte line folded into 4-byte words

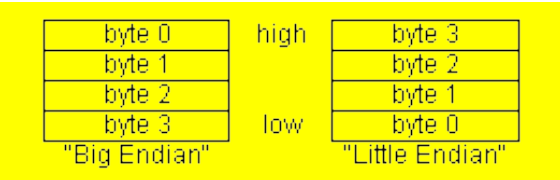
- Words must be aligned to 4 byte boundaries, and half-words to 2 byte boundaries

- Word addresses must be aligned, i.e., they must be multiple of 4
  → Both little-endian and big-endian memory addressing are supported



50

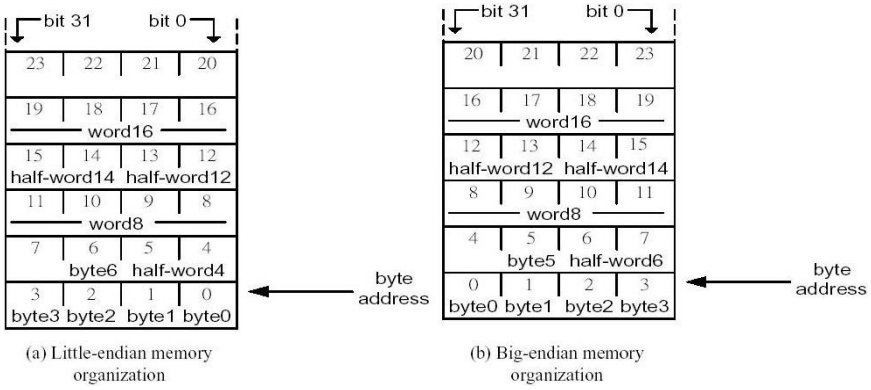
Storage of Data Structures



- Little Endian: 80x86, ARM
- Big Endian: mc680x0, SPARC, HP Precision, ARM
- Difference in "endian-ness" can be a problem when transferring data between machines

51

Memory Organization and Endianness

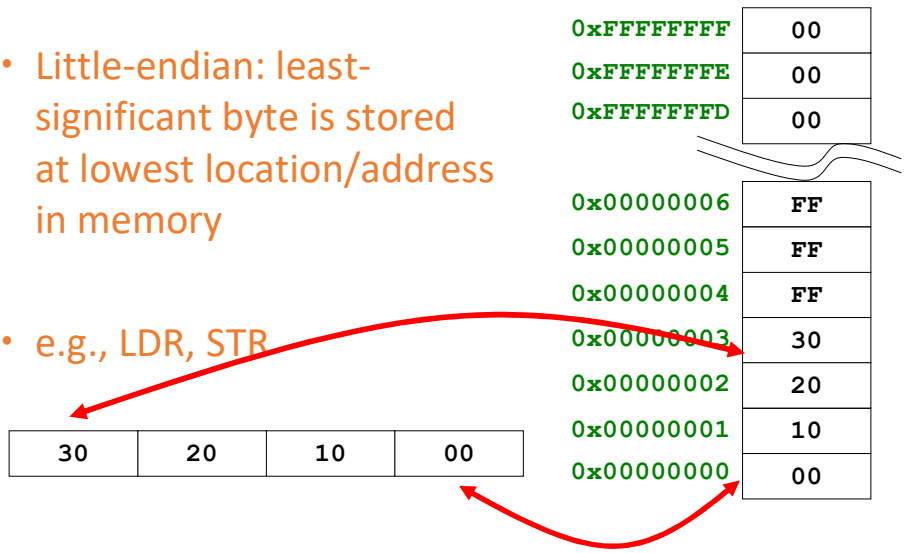


- Word, half-word alignment (xxxx00 or xxxxx0)
- ARM can be set up to access data in either *little-endian* or *big-endian* format, though they default to **little-endian**.

52

Little Endian Format Inside Memory

- Little-endian: least-significant byte is stored at lowest location/address in memory
- e.g., LDR, STR



53

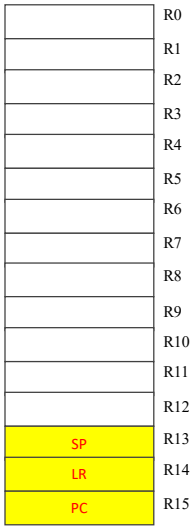
The Registers in ARM

- **ARM has 37 registers all of which are 32-bits long.**
  - **1** dedicated program counter
  - **1** dedicated current program status register
  - **5** dedicated saved program status registers
  - **30** general purpose registers
- **The current processor mode governs which of several banks is accessible. Each mode can access**
  - a particular set of **r0-r12** registers
  - a particular **r13** (the stack pointer, **sp**) and **r14** (the link register)
  - the program counter, **r15** (**pc**)
  - the current program status register, **cpsr**
- **Privileged modes (except System) can also access**
  - a particular spsr (saved program status register)

Internal

54

ARM Cortex-M Registers



- **Sixteen generic 32-bit registers**
  - Thirteen registers are general purpose
    - **Can hold data or address**
    - **Data may be byte, halfword or word (32 bits)**
  - Three special purpose registers
    - **R13 = stack pointer**
    - **R14 = link register**
      - To store return address during subroutine call
    - **R15 = program counter**

Open

55

Register Structure [1]

There are 15 additional general-purpose registers called the banked registers

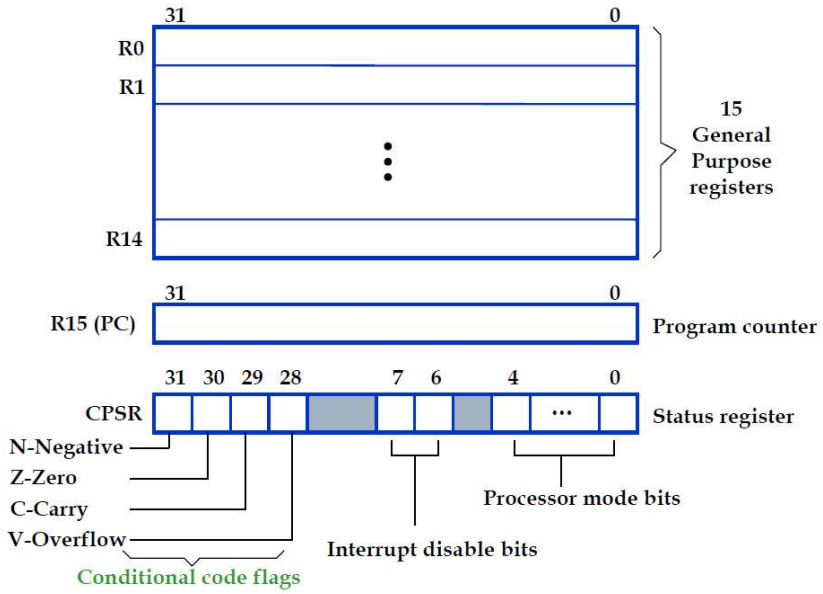
- They are duplicates of some of the R0 to R14 registers
- They are used when the processor switches into Supervisor or Interrupt modes of operation

Saved copies of the Status register are also available in the Supervisor and Interrupt modes

The use of processor mode bits and interrupt disable bits will be described in conjunction with input/output operations and interrupts

56

Register Structure [2]



57

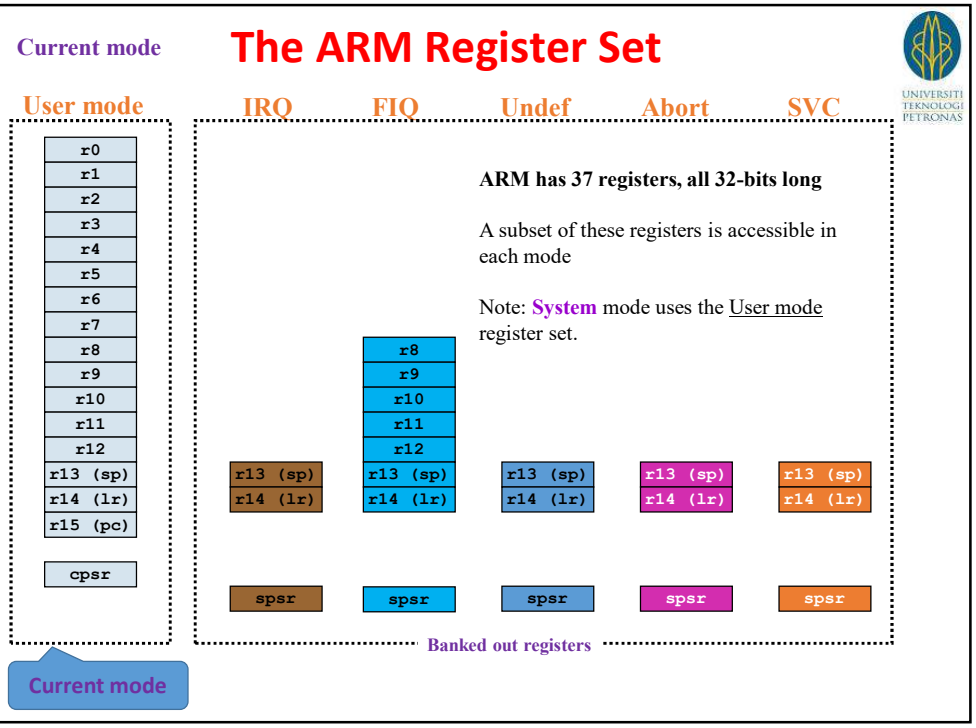
Processor Modes

- **ARM has seven (7) basic operating modes**
  - Each mode has access to its own stack space and a different subset of registers
  - Some operations can only be carried out in a privileged mode

	Mode	Description	
Exception modes	**Supervisor (SVC)**	Entered on reset and when a Supervisor call instruction (SVC) is executed	**Privileged modes**
	FIQ	Entered when a high priority (fast) interrupt is raised	
	IRQ	Entered when a normal priority interrupt is raised	
	Abort	Used to handle memory access violations	
	Undef	Used to handle undefined instructions	
	System	Privileged mode using the same registers as User mode	
	User	Mode under which most Applications / OS tasks run	**Unprivileged mode**

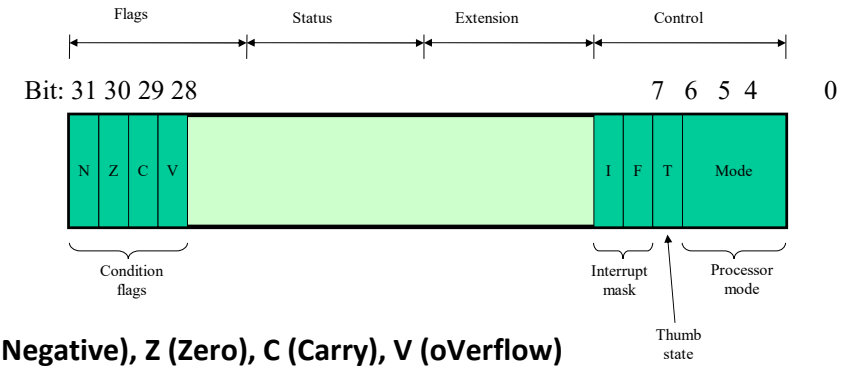
58

The ARM Register Set



59

Current Program Status Register (CPSR)

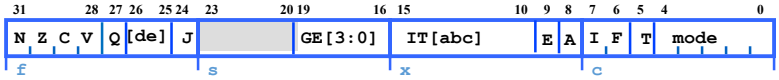


- **N (Negative), Z (Zero), C (Carry), V (oVerflow)**
- **mode – control processor mode**
- **T – control instruction set**
  - T = 1 – instruction stream is 16-bit Thumb instructions
  - T = 0 – instruction stream is 32-bit ARM instructions
- **I F – interrupt enables**

Internal

60

Current Program Status Registers (cpsr) [2]



- Condition code flags
  - N = Negative result from ALU
  - Z = Zero result from ALU
  - C = ALU operation Carried out
  - V = ALU operation oVerflowed
- Sticky Overflow flag - Q flag
  - Indicates if saturation has occurred
- IF THEN status bits – IT[de]
  - Controls conditional execution of Thumb instructions
- J bit
  - J = 1: Processor in Jazelle state
- SIMD Condition code bits – GE[3:0]
  - Used by some SIMD instructions
- IF THEN status bits – IT[abc]
  - Controls conditional execution of Thumb instructions
- E bit
  - E = 0: Data load/store is little endian
  - E = 1: Data load/store is big endian
- A bit
  - A = 1: Disable imprecise data aborts
- Interrupt Disable bits
  - I = 1: Disables IRQ
  - F = 1: Disables FIQ
- T bit
  - T = 0: Processor in ARM state
  - T = 1: Processor in Thumb state
- Mode bits
  - Specify the processor mode

61

Condition Codes

Bit	Name	Meaning after add or sub
N	negative	result is negative
Z	zero	result is zero
C	carry	unsigned overflow
V	overflow	signed overflow

- C set (i.e., C = 1) after an **unsigned** addition (e.g., ADDS in ARM) if the answer is wrong (cannot be represented by the given bits). If the answer is correct, C stays cleared (C=0).
- C cleared (i.e., C = 0) after an **unsigned** subtraction (e.g. SUBS in ARM) if the answer is wrong. If the answer is correct, C will be set (C=1).
- V set after a **signed** addition or subtraction if the answer is wrong

62

Data format

63

Addition Summary: ARM 32 bits



Let the 32-bit result R be the result of the 32-bit addition X+Y.

- **N bit** is set
  - if unsigned result is above 2^{31}-1 or
  - if signed result is negative.
  - $N = R_{31}$
- **Z bit** is set if result is zero
- **V bit** is set after a signed addition if result is incorrect
  - $V = X_{31}.M_{31}.\overline{R_{31}} + \overline{X_{31}}.\overline{M_{31}}.R_{31}$
- **C bit** is set after an unsigned addition if result is incorrect
  - $C = X_{31}.M_{31} + M_{31}.\overline{R_{31}} + \overline{R_{31}}.X_{31}$

64

Subtraction Summary: ARM 32 bits

Let the 32-bit result R be the result of the 32-bit subtraction X-Y.

- **N bit** is set
  - if unsigned result is above 2^{31}-1 or
  - if signed result is negative.
  - $N = R_{31}$
- **Z bit** is set if result is zero
- **V bit** is set after a signed subtraction if result is incorrect
  - $V = X_{31} \,\&\, \overline{M_{31}} \,\&\, \overline{R_{31}} \;\mid\; \overline{X_{31}} \,\&\, M_{31} \,\&\, R_{31}$
- **C bit** is clear after an unsigned subtraction if result is incorrect
  - $C = \overline{\overline{X_{31}} \,\&\, M_{31} \;\mid\; M_{31} \,\&\, R_{31} \;\mid\; R_{31} \,\&\, \overline{X_{31}}}$

65

Algorithm (Unsigned): ARM 32 bits

1. **Find values of both numbers interpreted as unsigned**
2. **Perform addition or subtraction**
3. **Does the result fit as an unsigned?**
   - **No -> addition C=1, subtraction C=0**
   - **Yes -> addition C=0, subtraction C=1**

For example: 255 + 5 = 260, C = 1 and the actual answer is 260-256 = 4

66

Algorithm (Signed): ARM 32 bits



1. **Find values of both numbers interpreted as signed**
2. **Perform addition or subtraction**
3. **Does the result fit as a signed?**
   - **No -> V=1**
   - **Yes -> V=0**

8-bit

Examples: **10 – 5 = 5 ; V=0**
-100 – 100 = -200 ; V=1

67

Data Format (8-bit): Review



- Unsigned Integers
  - All eight bits represent the magnitude of a number
    - bit 7 to bit 0
  - Range 00_H to FF_H (0_{10} to 255_{10})

NOTE: For ARM, we need to consider 32 bits, not just 8 bits.

68

Data Format (8-bit): Review



- Signed Integers
  - 2's Complement
    - bit 7 is sign bit (0 represents +ve; 1 represents –ve)
    - bit 6 to bit 0 represent magnitude
  - Positive numbers: 00_H to $7F_H$ (0_{10} to 127_{10})
  - Negative numbers: 80_H to FF_H (-1_{10} to -128_{10})

69

Unsigned and Signed Numbers involving 8-Bits Data Size



Unsigned Representation (0 to +255)

Binary	Hex	Decimal
0000 0000	00	0
…		
0111 1111	7F	+127
1000 0000	80	+128
…		
1111 1111	FF	+255

Signed Representation (-128 to +127)

2's complement:

Binary	Hex	Decimal
0000 0000	00	0
…		
0111 1111	7F	+127
1000 0000	80	-128
…		
1111 1111	FF	-1

70

Unsigned Addition and Carry (C) vs. Signed Addition and Overflow (OV): 8 bits



Unsigned Representation (0 to +255)

```
  0000 0011   (positive 3₁₀)
+ 0000 1001   (positive 9₁₀)
  0000 1100   (positive 12₁₀;
no carry; can represent
positive 12₁₀ using 8 bits)
```

C = 0

OV = 0

Z = 0

Signed Representation (-128 to +127)

```
  0000 0011   (positive 3₁₀)
+ 0000 1001   (positive 9₁₀)
  0000 1100   (no overflow;
can represent 1210 using 7
bit magnitude; the sign bit
conforms!)
```

NOTE: For ARM, we need to consider 32 bits, not just 8 bits.

71

Unsigned Addition and Carry (C) vs. Signed Addition and Overflow (OV): 8 bits



Unsigned Representation (0 to +255)

```
  0111 1111   (positive 127)
+ 0111 0011   (positive 115)
  1111 0010   (positive 242;
no carry; can represent
positive 242 using 8 bits)
```

C = 0

OV = 1

Z = 0

Signed Representation (-128 to +127)

```
  0111 1111   (positive 127)
+ 0111 0011   (positive 115)
  1111 0010   (overflow;
cannot represent 242 using 7
bit magnitude; the sign bit
is contradicting)
```

Note: +242 requires at least 9 bits: 0 1111 0010

NOTE: For ARM, we need to consider 32 bits, not just 8 bits.

72

Unsigned Addition and Carry (C) vs. Signed Addition and Overflow (OV): 8 bits



Unsigned Representation (0 to +255)

```
   1000 1111   (positive 143)
 + 1101 0011   (positive 211)
   0110 0010   (carry; cannot
represent positive 354 using
8 bits)
```

Signed Representation (-128 to +127)

```
  1000 1111   (negative 113)
+ 1101 0011   (negative  45)
  0110 0010   (overflow;
cannot represent negative 158
using 7 bits; the sign bit is
contradicting)
```

Note: -158 requires at least 9 bits: 1 0110 0010

C = 1

OV = 1

Z = 0

NOTE: For ARM, we need to consider 32 bits, not just 8 bits.

73

Unsigned Addition and Carry (C) vs. Signed Addition and Overflow (OV): 8 bits



Unsigned Representation (0 to +255)

```
  0011 1111   (positive 63)
+ 1101 0011   (positive 211)
  0001 0010   (carry; cannot
represent positive 274 using
8 bits)
```

Note: +274 requires at least 9 bits: 1 0001 0010

Signed Representation (-128 to +127)

```
  0011 1111   (positive 63)
  1101 0011   (negative 45)
  0001 0010 (positive 18, no
overflow; can be represented
by 7 bits; the sign bit
conforms)
```

C = 1

OV = 0

Z = 0

NOTE: For ARM, we need to consider 32 bits, not just 8 bits.

74

Unsigned Addition and Carry (C) vs. Signed Addition and Overflow (OV): 8 bits



Unsigned Representation

(0 to +255)

```
  1110 0100   (positive 228)
+ 1001 1100   (positive 156)
  1000 0000   (carry; cannot represent positive 384 using 8 bits)
```

Note: +384 requires at least 9 bits: 1 1000 0000

C = 1

OV = 0

Z = 0

Signed Representation

(-128 to +127)

```
  1110 0100   (negative 28)
+ 1001 1100   (negative 100)
  1000 0000   (negative 128, no overflow; can be represented by the sign bit ($b_7$) and magnitude bits ($b_6$ to $b_0$))
```

NOTE: For ARM, we need to consider 32 bits, not just 8 bits.

75